When Executability Is Not Faithfulness

Explaining a Two-Stage Teacher Failure and Its Repair

Case. `L2::bridge_join_0155#L2a`. A two-stage teacher produces an executable program that returns 1,137. Every online check passes, but the hidden benchmark answer is 63,118. The successful repair does not change the named buyer or the aggregation operator. It inserts the missing target population between the derived CPV set and the final count.

1 The question is about a derived population

The benchmark asks:

> How many procurement notices are listed under CPV codes that City of Doncaster Council has used before?

The named buyer is not the final population. It defines a source population from which a set of CPV codes must be derived. Those codes then define a new target population over the entire procurement snapshot. The intended computation is

$$\underbrace{\text{Doncaster records}}_{1{,}137} \longrightarrow \underbrace{\text{distinct CPV codes}}_{32} \longrightarrow \underbrace{\text{all records with those CPVs}}_{63{,}118} \longrightarrow \text{count}.$$

This distinction is easy to lose in natural language. "CPV codes that the council has used" is a set-valued condition inside the noun phrase modifying "procurement notices." Counting the source records or the codes answers a related but different question.

2 Stage 1 identifies the bridge but chooses the wrong endpoint

The Stage 1 model, `gpt-5.4-nano`, is asked to convert the question into a typed computation program without answering it. Its stored response is equivalent to

```
A = filter_records(buyer = City of Doncaster Council)
B = distinct(A, field = CPV code)
C = count(B)
return C
```

The response gets two important facts right. It grounds City of Doncaster Council as the buyer, and it understands that the buyer's records produce a set of CPV codes. The final type is wrong, however. The question requests a count of records, whereas `C` counts the intermediate entity set.

Stage 1 also reports that "used before" lacks a temporal anchor. In this benchmark family the phrase denotes codes observed for the buyer in the fixed graph snapshot; it does not define a per-record temporal comparison. The model therefore introduces an ambiguity that is not part of the executable benchmark contract.

Stage 1 failure. The model discovers the source-to-CPV relation but treats the derived CPV set as the answer population. It also over-interprets a benchmark surface phrase as a missing temporal constraint.

3 Why Stage 2 is invoked

The intent program can be compiled, but the committed routing code applies a semantic fast-path check before accepting it. For this row the check returns

`count_target_is_entity_set_for_record_question`.

This veto prevents the deterministic compiler from being the final planner and invokes the configured lean Stage 2 planner, `grok-4-1-fast-non-reasoning`. Stage 2 sees the original question, the complete Stage 1 briefing, retrieved schema context, and the graph-planning rules.

The historical compact corpus preserves the Stage 1 briefing and the normalized rejected graph, but not an explicit route tag or the raw provider request and response bytes. The two-stage route is therefore established by applying the committed routing code to the stored Stage 1 object. The same code emits the full reconstructed Stage 2 prompt included in the companion prompt bundle.

4 Stage 2 remains executable but incomplete

The normalized rejected graph contains a buyer-filtered record set and a derived entity variable:

```
A = record_set(buyer = City of Doncaster Council)
B = entity_set(depends_on = A, role = none)
return count(B)
```

Stage 2 does not create the required target record set. Its entity role is also left unspecified. Execution consequently remains on the source side of the bridge and returns 1,137, the size of the Doncaster record population, rather than the number of records sharing Doncaster's CPVs.

This is not a parser failure. The graph has valid fields, a supported count operation, a grounded buyer, a nonempty execution, and a numeric answer. It is therefore a particularly informative failure: the program is operationally valid while its population semantics are incomplete.

Contract	Verdict	What it establishes
Schema and grounding	Pass	The fields and named buyer are executable
Operation support	Pass	Count is supported on the returned object
Execution and evidence	Pass	The program runs and returns nonempty evidence
Output consistency	Pass	The result has the expected numeric shape
Offline oracle agreement	Fail	1,137 does not equal 63,118
Constraint faithfulness	Exposed by audit	The target record population is absent

Table 1: Why an online success and an offline semantic failure can coexist.

5 The verifier does not have contradictory opinions

The online and offline verdicts answer different questions. The online runtime asks whether the submitted action is grounded, executable, supported, and internally consistent. The offline evaluator additionally asks whether its result agrees with a benchmark oracle.

Let p be the proposed graph program and $X(p)$ its deterministic execution. The observed outcome is

$$f_{\text{execute}}(p) = 1, \qquad f_{\text{release}}(X(p)) = 1, \qquad f_{\text{oracle}}(X(p), y^*) = 0.$$

There is no contradiction. Executability is a weaker claim than faithfulness to the full question. The result illustrates the paper's execution–faithfulness gap:

$$\text{successful execution} \not\Rightarrow \text{correct reasoning population}.$$

6 Repair changes the graph, not the answer string

The offline controller uses the hidden oracle only to decide that a repair attempt may be collected. The repair model is told that the submitted answer failed hidden validation and that the reference answer is withheld. It sees 1,137 as the submitted answer but does not see 63,118.

The accepted repair is

```
a1 = record_set(buyer = City of Doncaster Council)
b1 = entity_set(role = cpv, depends_on = a1)
c1 = record_set(depends_on = b1)
relation a1 -> b1, bind CPV
relation b1 -> c1, bind CPV
return count(c1)
```

The critical change is `c1`. It turns the derived codes into a filter over a new record population. Deterministic replay produces

Variable	Kind	Size	Meaning
`a1`	Record set	1,137	Doncaster's source records
`b1`	Entity set	32	Distinct CPV codes from `a1`
`c1`	Record set	63,118	All records whose CPV is in `b1`

The repaired answer is therefore 63,118. The same online checks pass again, and the result now also agrees with the hidden oracle.

Repair attribution. The improvement cannot be attributed to a new buyer grounding, additional literals, answer copying, or a different aggregation operator. It is attributable to one inspectable structural change: inserting the CPV-bound target record population.

7 What this phenomenon shows

First, stronger generation alone is insufficient. Both teacher stages can preserve many local details while losing the global answer population. Second, a verifier has bounded authority. A runtime verifier should not be described as proving semantic correctness when it has no independent oracle for the intended population. Third, an executable environment makes improvement attributable. The proposal, intermediate sets, feedback boundary, repair, and changed outcome can all be inspected independently.

The claim supported by this case is consequently narrow but useful:

> The environment does not create or certify improvement. It makes reasoning actions, their intermediate populations, and the feedback pathway observable enough to locate both a gain and the boundary of verification.

8 Reproduction and evidence status

All stored objects are joined by `L2::bridge_join_0155#L2a`.

Artifact	Preserved evidence
`data/qa/cicada_core_v4/train.jsonl`	Question, latent specification, oracle
`data/qa/teacher_full_v1/traces.jsonl`	Briefing, rejected graph, 1,137, verdict
`data/qa/teacher_full_v1/hard_negatives.jsonl`	Offline rejected example
`data/qa/teacher_full_v1/repair_sft.jsonl`	Bounded feedback and accepted repair
`data/qa/teacher_full_v1/dpo_pairs.jsonl`	Chosen and rejected plans
`doncaster_two_stage_teacher_prompt_bundle.txt`	Full readable prompt reconstruction
`doncaster_two_stage_teacher_prompt_bundle.json`	Structured prompts, schemas, and evidence labels

The historical rows support the question, parsed Stage 1 output, normalized rejected graph, initial answer, oracle mismatch, and accepted repair. The complete Stage 1 and Stage 2 messages are emitted again

by the committed prompt builders. Because raw historical provider messages were compacted, they are labelled reconstructions rather than byte-identical API transcripts. The $1{,}137 \to 32 \to 63{,}118$ transition is recomputed on the versioned graph with the accepted stored plan.